Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Laboratory Work 1:

Caesar Cipher & Caesar Cipher with Keyword Permutation

Elaborated: st. gr. FAF-243
Șoimu Ionuț

Verified: asist. univ.
Zaica Maia

Chișinău – 2026

Contents

1 Introduction and Objectives **2**

2 Short Theory **2**
2.1 The Classical Caesar Cipher 2
2.1.1 Encryption and Decryption Formulas 2
2.1.2 Key Space and Security Vulnerabilities 3
2.2 The Caesar Cipher with a Permutation (Two Keys) 3
2.2.1 Permuted Alphabet Construction 3
2.2.2 Encryption and Decryption over the Permuted Domain 3
2.2.3 Key Space Analysis and Cryptanalytic Resilience 3

3 Implementation Details **4**
3.1 Architecture and Romanian Alphabet Representation 4
3.2 Code Fragments and Explanations 4
3.2.1 Alphabet Definition and Explicit Mapping 4
3.2.2 Diacritic Normalization and Cedilla Compatibility 5
3.2.3 Strict Input Validation and Error Reporting 5
3.2.4 Building the Permuted Alphabet 6
3.2.5 Encryption and Decryption Algorithms 6

4 Results and Demonstration **7**
4.1 Task 1.1: Single-Key Caesar Cipher Execution 7
4.1.1 Operational Verification and Hand Calculation 7
4.2 Task 1.1: Error Handling and Input Validation 8
4.3 Task 1.2: Two-Key Caesar Cipher with Permutation 9
4.3.1 Permuted Alphabet Generation and Hand-Check Verification 9
4.4 Task 1.2: Error Handling for Keyword and Shift Keys 10

5 Conclusions **12**

6 Source Code Repository **12**

1 Introduction and Objectives

Classical cryptography provides foundational principles for confidential communication. Among historical ciphers, monoalphabetic substitution ciphers represent the earliest attempts to obscure plaintext by systematically replacing characters according to fixed rules.

The objectives of this laboratory work are:

1. **Understand the mathematical foundations** and operational mechanics of the Caesar cipher.
2. **Analyze the cryptographic weakness** of the Caesar cipher stemming from its trivial key space and vulnerability to brute-force attacks.
3. **Implement keyword-based alphabet permutation** to significantly expand the key space and evaluate its effect on security.
4. **Adapt classical algorithms to non-English alphabets**, specifically the 31-letter Romanian alphabet, enforcing strict indexing rules (Table 2) without relying on language-dependent ASCII/Unicode codepoint shifts.
5. **Enforce robust input validation**, handling edge cases such as diacritic normalization (comma-below vs. cedilla), whitespace stripping, rejected character identification, and loop re-prompting.

2 Short Theory

2.1 The Classical Caesar Cipher

The Caesar cipher is an additive monoalphabetic substitution cipher attributed to Julius Caesar, who historically utilized a fixed shift of $k = 3$ to communicate military intelligence. Over an arbitrary finite alphabet Σ of cardinality $n = |\Sigma|$, each symbol $m_i \in \Sigma$ is assigned a unique integer index $x_i \in \mathbb{Z}_n = \{0, 1, \ldots, n-1\}$ through a bijective encoding function $\text{enc} : \Sigma \to \mathbb{Z}_n$.

The secret key k is an integer chosen from the non-zero elements of the additive group $\mathbb{Z}_n$:

$$k \in \mathscr{K} = \{1, 2, \ldots, n-1\}. \tag{1}$$

The identity key $k = 0$ is explicitly excluded since it maps every plaintext character directly to itself ($e_0(x) = x$), providing zero obfuscation.

2.1.1 Encryption and Decryption Formulas

For a plaintext message consisting of numeric codes $(x_1, x_2, \ldots, x_N)$, encryption e_k is computed using modular addition:

$$c_i = e_k(x_i) = (x_i + k) \pmod{n}. \tag{2}$$

Decryption d_k is the inverse transformation under modular arithmetic, subtracting the secret key k:

$$x_i = d_k(c_i) = (c_i - k) \pmod{n}. \tag{3}$$

Because differences $(c_i - k)$ may produce negative integer values, the modulo operation must wrap around into the non-negative residue domain $[0, n-1]$:

$$(c_i - k) \pmod{n} \equiv ((c_i - k) \bmod n + n) \bmod n. \tag{4}$$

2.1.2 Key Space and Security Vulnerabilities

For the Romanian alphabet with $n = 31$, the key space is:

$$|\mathscr{K}_{\text{Caesar}}| = n - 1 = 30. \tag{5}$$

A key space of only 30 possible keys renders the cipher trivially vulnerable to an **exhaustive key search (brute-force attack)**. An adversary intercepting ciphertext c can compute all 30 candidate plaintexts $d_1(c), d_2(c), \ldots, d_{30}(c)$ instantaneously. Because natural language possesses high redundancy, exactly one candidate key yields readable Romanian text.

2.2 The Caesar Cipher with a Permutation (Two Keys)

To defend against trivial exhaustive search, the natural ordering of the alphabet can be permuted using a secondary secret key: a keyword k_2. The resulting cryptosystem utilizes two secret keys:

- **Key 1 (k_1):** The numeric Caesar shift value, $k_1 \in \{1, 2, \ldots, n-1\}$.
- **Key 2 (k_2):** A keyword string consisting of Romanian letters with length $|k_2| \geq 7$.

2.2.1 Permuted Alphabet Construction

The permuted alphabet $\Sigma_{\text{perm}} = (\sigma'_0, \sigma'_1, \ldots, \sigma'_{n-1})$ is assembled deterministically:

1. Convert k_2 to uppercase and filter out spaces or diacritic variations.
2. Extract the distinct letters of k_2 in their order of initial appearance, discarding duplicate occurrences.
3. Append the remaining letters of the original alphabet Σ that were not in k_2, preserving their original canonical ordering (Table 2).

2.2.2 Encryption and Decryption over the Permuted Domain

Let $\text{pos}_{\text{perm}}(s)$ denote the index of character s within Σ_{perm}. The cryptographic transformations are defined as:

$$c_i = \Sigma_{\text{perm}} \left[(\text{pos}_{\text{perm}}(m_i) + k_1) \pmod{n} \right], \tag{6}$$

$$m_i = \Sigma_{\text{perm}} \left[(\text{pos}_{\text{perm}}(c_i) - k_1) \pmod{n} \right]. \tag{7}$$

2.2.3 Key Space Analysis and Cryptanalytic Resilience

The number of distinct permutations of an n-element alphabet is given by the factorial $n!$. For $n = 31$:

$$31! \approx 8.2228 \times 10^{33}. \tag{8}$$

Combined with the 30 possible shifts of k_1, the theoretical key pair space is:

$$|\mathscr{K}_{\text{total}}| = 30 \times 31! \approx 2.4668 \times 10^{35}. \tag{9}$$

While keyword-derived permutations span a subset of all 31! permutations (due to dictionary word constraints), the effective search space far exceeds 10^{15} candidates, entirely neutralizing brute-force enumeration.

However, **the cipher remains a monoalphabetic substitution cipher**. A fixed plaintext character x is mapped to the exact same ciphertext character c throughout the entire message. Therefore:

- The unigram, bigram, and trigram frequency distributions of the source language are fully preserved in the ciphertext.
- In Romanian, vowels (`E`, `I`, `A`) and consonants (`R`, `T`, `N`) occur with distinct probability peaks.
- A cryptanalyst can bypass the keyword and shift entirely by performing **frequency analysis**, matching high-frequency ciphertext symbols to known Romanian language statistics.

3 Implementation Details

3.1 Architecture and Romanian Alphabet Representation

The implementation was constructed in Python with a clean, modular architecture separated into distinct components:

- `romanian_alphabet.py`: Canonical definition of Table 2, diacritic normalizers, permutation builders, and validation routines.
- `caesar_cipher.py`: Task 1.1 single-key Caesar cipher implementation with interactive user prompts and re-prompting loops.
- `caesar_permuted.py`: Task 1.2 two-key Caesar cipher implementation with permuted alphabet generation and inspection table formatting.
- `main.py`: Unified CLI and interactive interface supporting parameter parsing, automated demonstrations, and user workflows.

Table 1 outlines the canonical Table 2 encoding prescribed by the laboratory specifications for $n = 31$.

Table 1: Canonical Romanian Alphabet Letter Encoding ($n = 31$, Table 2)

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
Code	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Letter	N	O	P	Q	R	S	Ș	T	Ț	U	V	W	X	Y	Z	–
Code	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	–

3.2 Code Fragments and Explanations

3.2.1 Alphabet Definition and Explicit Mapping

The lab strictly requires that built-in ASCII/Unicode code point arithmetic must not be used to compute the Caesar shift. Instead, an explicit list and bidirectional dictionary mappings are established:

```
ROMANIAN_ALPHABET: List[str] = [
    'A', 'Ă', 'Â', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I',
    'Î', 'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R', 'S',
    'Ș', 'T', 'Ț', 'U', 'V', 'W', 'X', 'Y', 'Z'
]
ALPHABET_SIZE: int = len(ROMANIAN_ALPHABET)  # 31

CHAR_TO_INDEX: Dict[str, int] = {char: idx for idx, char in enumerate(
    ROMANIAN_ALPHABET)}
INDEX_TO_CHAR: Dict[int, str] = {idx: char for idx, char in enumerate(
    ROMANIAN_ALPHABET)}
```

Comment: This defines the exact domain $\mathbb{Z}_{31}$. Mapping lookup is $O(1)$ and guarantees that letter substitutions strictly follow Table 2 rather than ASCII offsets.

3.2.2 Diacritic Normalization and Cedilla Compatibility

Romanian keyboards and typography frequently interchange comma-below letters (Ș, Ț – Unicode U+0218, U+021A) with Turkish-style cedilla variants (Ş, Ţ – Unicode U+015E, U+0162). To ensure compatibility across different operating systems and keyboards, an explicit normalizer was implemented:

```
CEDILLA_MAPPING: Dict[str, str] = {
    'Ş': 'Ș', 'ş': 'ș',
    'Ţ': 'Ț', 'ţ': 'ț',
}

def normalize_text(text: str) -> str:
    """Normalizes cedillas to comma-below and converts to uppercase."""
    normalized_chars = [CEDILLA_MAPPING.get(c, c).upper() for c in text]
    return "".join(normalized_chars)
```

Comment: All input characters are converted to standard comma-below representations before any membership validation or cryptographic transformation occurs.

3.2.3 Strict Input Validation and Error Reporting

The laboratory specification requires rejecting invalid keys or non-alphabet characters with a clear message indicating the allowed range and the exact rejected character.

```
def validate_key(key_input: str) -> Tuple[bool, Optional[str], int]:
    try:
        k = int(key_input.strip())
    except ValueError:
        return False, f"Error: Invalid key '{key_input}'. Must be integer 1..30.",
    -1
    if 1 <= k <= (ALPHABET_SIZE - 1):
        return True, None, k
    return False, f"Error: Key {k} out of range. Allowed: 1..30 inclusive.", -1

def validate_text(raw_text: str) -> Tuple[bool, Optional[str], str]:
    if not raw_text:
        return False, "Error: Text input cannot be empty.", ""
    normalized = normalize_text(raw_text)
    cleaned_chars = []
    for idx, ch in enumerate(normalized):
        if ch.isspace():
            continue
        if ch in CHAR_TO_INDEX:
            cleaned_chars.append(ch)
        else:
            orig = raw_text[idx]
            return False, (
                f"Error: Invalid character '{orig}' (U+{ord(orig):04X}) at pos {idx
     + 1}.\n"
                f"Allowed characters: Romanian alphabet letters (A-Z, Ă, Â, Î, Ș, Ț
    ) and spaces."
            ), ""
    return True, None, "".join(cleaned_chars)
```

Comment: Spaces are permitted in user input and stripped automatically. Any foreign symbol (numbers, punctuation, symbols) is caught immediately, highlighting the 1-indexed character position and its Unicode codepoint.

3.2.4 Building the Permuted Alphabet

For Task 1.2, the permuted alphabet is constructed by deduplicating the keyword and appending the remaining Romanian letters:

```
def build_permuted_alphabet(keyword: str) -> List[str]:
    normalized_kw = normalize_text(keyword)
    permuted: List[str] = []
    seen = set()

    for ch in normalized_kw:
        if ch in CHAR_TO_INDEX and ch not in seen:
            seen.add(ch)
            permuted.append(ch)

    for ch in ROMANIAN_ALPHABET:
        if ch not in seen:
            seen.add(ch)
            permuted.append(ch)

    assert len(permuted) == ALPHABET_SIZE
    return permuted
```

Comment: Preserves the insertion order of unique keyword characters, guarantees that all 31 Romanian alphabet symbols are present exactly once, and maintains a linear $O(n)$ complexity.

3.2.5 Encryption and Decryption Algorithms

Both Task 1.1 and Task 1.2 transformations map characters to indices, apply modular shifts, and map indices back to the corresponding alphabet:

```
def encrypt_caesar(plaintext: str, key: int) -> str:
    return "".join(INDEX_TO_CHAR[(CHAR_TO_INDEX[c] + key) % ALPHABET_SIZE] for c in
     plaintext)

def decrypt_caesar(ciphertext: str, key: int) -> str:
    return "".join(INDEX_TO_CHAR[(CHAR_TO_INDEX[c] - key) % ALPHABET_SIZE] for c in
     ciphertext)

def encrypt_caesar_permuted(plaintext: str, key1: int, permuted: List[str]) -> str:
    char_to_pos = {char: idx for idx, char in enumerate(permuted)}
    return "".join(permuted[(char_to_pos[c] + key1) % ALPHABET_SIZE] for c in
    plaintext)

def decrypt_caesar_permuted(ciphertext: str, key1: int, permuted: List[str]) -> str
    :
    char_to_pos = {char: idx for idx, char in enumerate(permuted)}
    return "".join(permuted[(char_to_pos[c] - key1) % ALPHABET_SIZE] for c in
    ciphertext)
```

Comment: Python's native modulo operator `%` implements mathematical floored division, ensuring that `(y - key) % 31` always produces a positive residue in the range $[0, 30]$.

4 Results and Demonstration

4.1 Task 1.1: Single-Key Caesar Cipher Execution

4.1.1 Operational Verification and Hand Calculation

Consider encrypting the Romanian test string:

`"Cifrul Cezar cu caractere românești"`

with shift key $k = 5$.

1. **Preprocessing:** Spaces are stripped, cedillas converted, and text cast to uppercase:

   `"CIFRULCEZARCUCARACTEREROMÂNEȘTI"`

2. **Letter Transformation Steps (Excerpt):**
   - `C`: Code 4 $\Longrightarrow$ $(4+5) \bmod 31 = 9$ $\Longrightarrow$ `H`
   - `I`: Code 10 $\Longrightarrow$ $(10+5) \bmod 31 = 15$ $\Longrightarrow$ `M`
   - `F`: Code 7 $\Longrightarrow$ $(7+5) \bmod 31 = 12$ $\Longrightarrow$ `J`
   - `R`: Code 20 $\Longrightarrow$ $(20+5) \bmod 31 = 25$ $\Longrightarrow$ `U`
   - `U`: Code 25 $\Longrightarrow$ $(25+5) \bmod 31 = 30$ $\Longrightarrow$ `Z`
   - `L`: Code 14 $\Longrightarrow$ $(14+5) \bmod 31 = 19$ $\Longrightarrow$ `Q`
   - `Z`: Code 30 $\Longrightarrow$ $(30+5) \bmod 31 = 4$ $\Longrightarrow$ `C` (Wrap-around)
   - `Ș`: Code 22 $\Longrightarrow$ $(22+5) \bmod 31 = 27$ $\Longrightarrow$ `W`
   - `Ț`: Code 24 $\Longrightarrow$ $(24+5) \bmod 31 = 29$ $\Longrightarrow$ `Y`

3. **Ciphertext Produced:** `"HMJUZQHÎCDUHZHDUDHXÎUÎUȘRFSÎ"`

4. **Decryption Verification:** Applying the decryption function $d_5(y) = (y-5) \bmod 31$ to each character of the ciphertext restores the identical original plaintext string.

Figure 1 illustrates the terminal session in the Kitty terminal running `caesar_cipher.py`, displaying the canonical Romanian alphabet mapping table, successful encryption of the phrase, and reciprocal decryption verifying round-trip correctness.

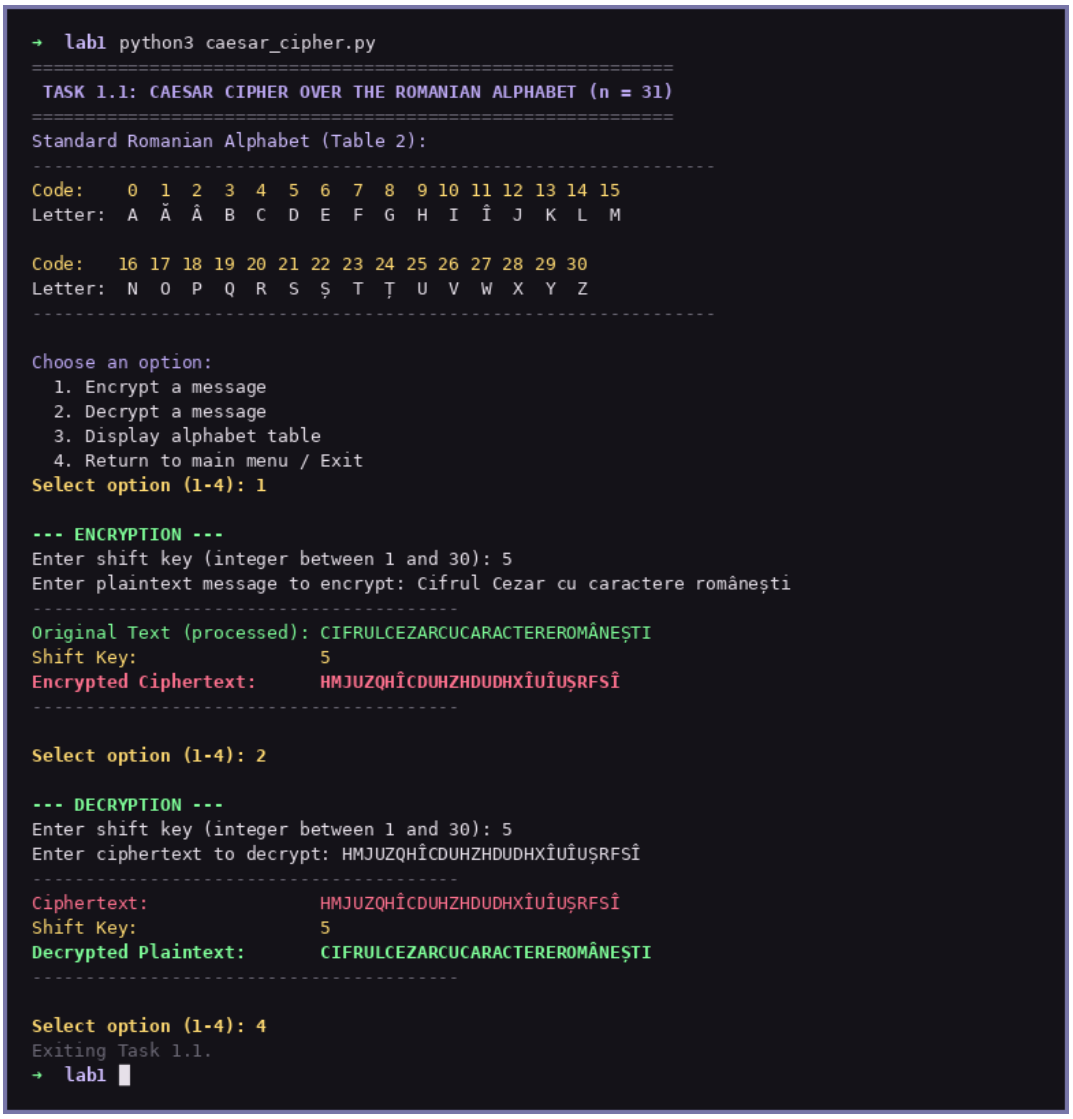


Figure 1: Task 1.1 terminal capture: Encryption and decryption execution over Romanian alphabet with diacritics.

4.2 Task 1.1: Error Handling and Input Validation

The implementation strictly validates user input and displays informative error diagnostics before requesting the value again:

- **Out-of-range Shift Key:** Inputting $k = 35$ produces a message stating the allowed interval $1 \leq k \leq 30$.
- **Non-numeric Key:** Entering non-integer string values such as `"abc"` is caught and rejected gracefully.
- **Illegal Characters:** Entering `"Salut 2026!"` isolates character `'2'` at position 7 as non-Romanian and displays the allowed alphabet range before re-prompting.
- **Recovery:** Valid text `"Securitate Informațională"` with key $k = 7$ is successfully processed into `"YJFÂYNĂĂGNPUTVRĂȚNTUTĂF"`.

Figure 2 documents this validation behavior and prompt recovery sequence in the terminal.

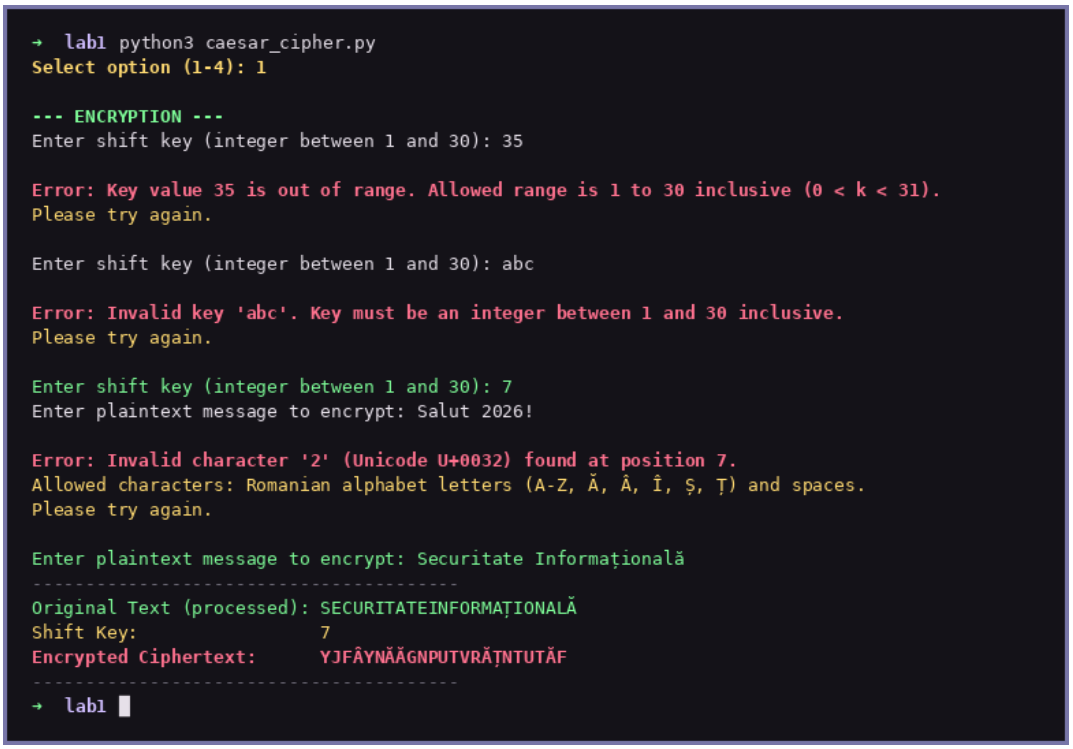


Figure 2: Task 1.1 terminal capture: Diagnostic messages for out-of-range keys, non-numeric inputs, and illegal character positions.

4.3 Task 1.2: Two-Key Caesar Cipher with Permutation

4.3.1 Permuted Alphabet Generation and Hand-Check Verification

Using keyword $k_2 =$ `"CRIPTOGRAFIE"` ($|k_2| = 12 \geq 7$):

1. **Unique Keyword Characters:** `C, R, I, P, T, O, G, A, F, E` (subsequent occurrences of `R` and `I` are discarded).
2. **Remaining Letters Appended:** Ă, Â, B, D, H, Î, J, K, L, M, N, Q, S, Ș, Ț, U, V, W, X, Y, Z.
3. **Resulting Permuted Alphabet (Table 2):**

Table 2: Permuted Romanian Alphabet for Keyword `"CRIPTOGRAFIE"`

Pos	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Letter	C	R	I	P	T	O	G	A	F	E	Ă	Â	B	D	H	Î
Pos	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	–
Letter	J	K	L	M	N	Q	S	Ș	Ț	U	V	W	X	Y	Z	–

Encrypting `"CALCULATOARE SI MICROELECTRONICA"` with shift $k_1 = 3$:

- `C`: Pos 0 $\Longrightarrow$ $(0+3) \bmod 31 = 3$ $\Longrightarrow$ `P`
- `A`: Pos 7 $\Longrightarrow$ $(7+3) \bmod 31 = 10$ $\Longrightarrow$ `Ă`
- `L`: Pos 18 $\Longrightarrow$ $(18+3) \bmod 31 = 21$ $\Longrightarrow$ `Q`
- `C`: Pos 0 $\Longrightarrow$ 3 $\Longrightarrow$ `P`
- `U`: Pos 25 $\Longrightarrow$ $(25+3) \bmod 31 = 28$ $\Longrightarrow$ `X`

Yielding the ciphertext: "PĂQPXQĂAFĂTBUOSOPTFBQBPATFȘOPĂ".

Figure 3 demonstrates the execution of `caesar_permuted.py`, displaying the permuted alphabet for manual verification followed by two-key encryption and decryption.

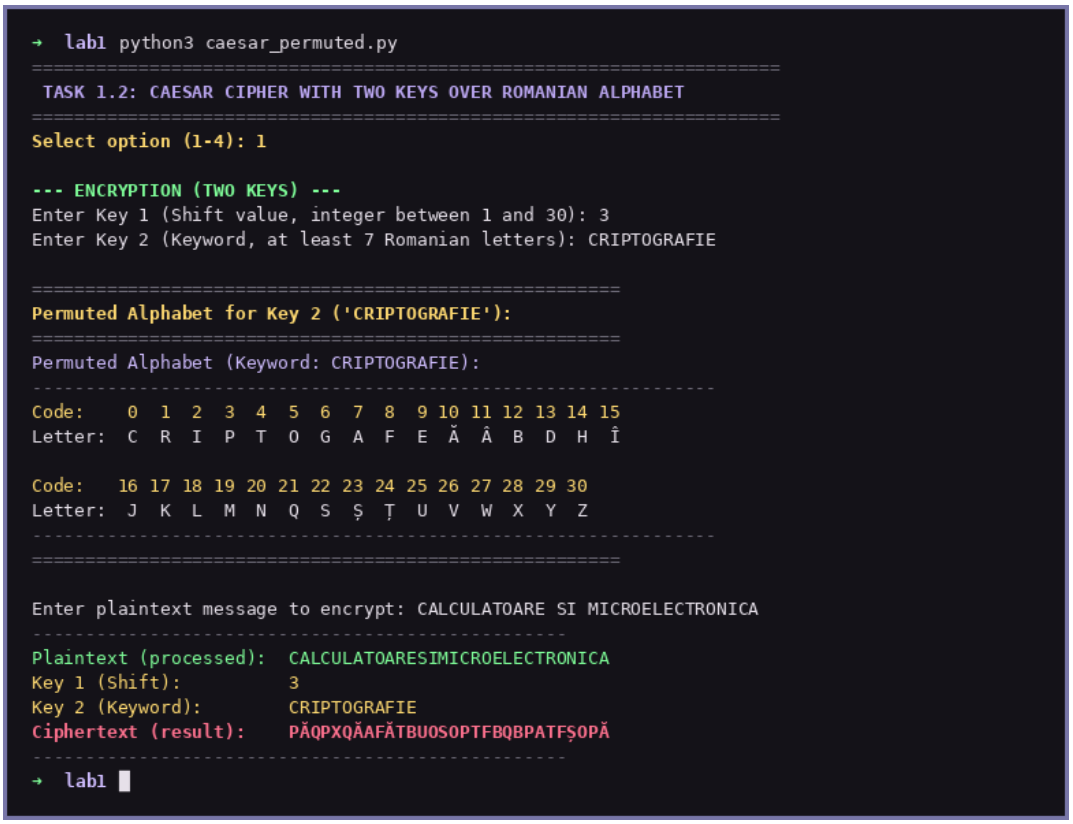


Figure 3: Task 1.2 terminal capture: Permuted alphabet displayed for verification, followed by two-key encryption and decryption.

4.4 Task 1.2: Error Handling for Keyword and Shift Keys

Figure 4 captures the error handling for Task 1.2 inputs:

- **Shift Boundary Enforcement:** Key $k_1 = 0$ is rejected with a range notice ($1 \leq k_1 \leq 30$).
- **Keyword Length Threshold:** Keyword `"CEZAR"` (length 5) is rejected because the task requires $|k_2| \geq 7$.
- **Keyword Character Validation:** Keyword `"CRIPTO#SEC"` is rejected due to illegal character `'#'` at position 7.
- **Recovery:** A valid keyword `"SECRETLAB"` is accepted, and its permuted alphabet is printed.



Figure 4: Task 1.2 terminal capture: Validation errors for shift bounds, short keywords, and illegal symbols.

5 Conclusions

Through this laboratory work, I implemented and evaluated two classical substitution ciphers adapted to the Romanian language: the standard single-key Caesar cipher and the keyword-permuted two-key Caesar cipher.

- **Key Space and Brute-Force Resilience:** The standard Caesar cipher over the 31-letter Romanian alphabet possesses only 30 possible keys, making exhaustive search trivial by hand or on a computer. In contrast, introducing a keyword permutation expands the theoretical permutation space to $31! \times 30 \approx 2.47 \times 10^{35}$, completely eliminating brute-force search as a viable attack vector.
- **Vulnerability to Frequency Analysis:** Despite the massive expansion in key space, the two-key Caesar cipher remains a monoalphabetic substitution cipher. Because the mapping from plaintext to ciphertext letters is 1-to-1 and stationary, language statistics (such as the high frequency of Romanian letters `E`, `I`, `A`, `R`, and `T`) remain intact in the ciphertext. Thus, an attacker armed with ciphertext of sufficient length can recover the plaintext using frequency analysis without discovering the secret keyword.
- **Alphabet Handling and Engineering Considerations:** Restricting the cipher to Table 2 required avoiding ASCII/Unicode arithmetic shortcuts and implementing custom index lookups. Handling Romanian diacritics also required normalising cedilla characters (Ş, Ţ) to standard comma-below characters (Ș, Ț).

In conclusion, this work demonstrated that expanding key space alone does not ensure cryptographic security if structural statistical patterns are preserved. Truly secure symmetric encryption requires diffusion and confusion principles (as found in polyalphabetic ciphers and modern block ciphers like AES).

6 Source Code Repository

The complete source code and report build files are organized in the project repository:

GitHub Repository Link:
https://github.com/Lucariowu/cslabs